

Test 02 Practice - Potential Solutions

1. (10 PTS.) These questions deal with transforming a VList collection; consider the following book record:

```
record Book(String title, int pages, int year) { }

var books = VList.of(
    new Book("The Martian", 220, 2019),
    new Book("Project Hail Mary", 410, 2023),
    new Book("Biological War: A Scenario", 432, 2026),
    new Book("Dust", 416, 2013)
);
```

(a) (3 PTS.) use filter to produce a list containing only books with at least 300 pages

```
var longBooks =
    books.filter(b -> b.pages() >= 300);
```

(b) (3 PTS.) use map to produce a new VList<Book> where each book has the same title and year, but its number of pages is increased by 10

```
var expandedBooks = books.map(b ->
    new Book(b.title(), b.pages() + 10, b.year()));
```

(c) (4 PTS.) use both filter and map to produce the titles of books published in 2020 or later

```
var recentTitles = books
    .filter(b -> b.year() >= 2020)
    .map(b -> b.title());
```

2. (5 PTS.) These questions also deal with VList.

(a) (3 PTS.) state both the contents and the inferred type for xs, ys, and zs

xs: VList<Book>; contains Project Hail Mary, Biological War: A Scenario, and Dust, in that order.

ys: VList<Integer>; [220, 410, 432, 416].

zs: VList<String>; ["The Martian", "Project Hail Mary", "Biological War: A Scenario", "Dust"].

(b) (2 PTS.) does filter modify books?

No. It produces another VList; the original books value remains unchanged.

3. (7 PTS.) This question deals with functional interfaces.

```
Function<Integer, String> f = n -> "value=" + n;  
Predicate<String> p      = s -> s.length() < 6;
```

(a) (2 PTS.)

f: parameter Integer, return String.

p: parameter String, return boolean.

(b) (3 PTS.)

```
f.apply(7)           // "value=7"  
p.test("abc")        // true  
p.test("abcdef")     // false
```

(c) (2 PTS.)

Function<A,B> takes an A and produces a B using apply. Predicate<A> tests an A and produces a boolean using test.

4. (6 PTS.) This question deals with Lambda (λ) expressions.

(a) (2 PTS.)

```
Predicate<Integer> negative = n -> n < 0;
```

(b) (2 PTS.)

```
Function<String, Integer> lengthPlusOne =  
    s -> s.length() + 1;
```

(c) (2 PTS.)

```
Function<Integer, String> parity =  
    n -> n % 2 == 0 ? "even" : "odd";
```

5. (5 PTS.) This is more of an object modeling question.

```
sealed interface Parcel permits Box, Envelope {  
    double weight();  
}
```

```
record Box(double weight, double volume)  
    implements Parcel { }
```

```
record Envelope(double weight, boolean certified)  
    implements Parcel { }
```

The generated weight() accessor in each record satisfies the method required by Parcel.

6. (7 PTS.) This question deals with the Maybe type.

(a) (3 PTS.)

```
Maybe<Integer> firstNegative(VList<Integer> values) {  
    for (var value : values) {  
        if (value < 0) {  
            return Maybe.of(value);  
        }  
    }  
    return Maybe.none();  
}
```

(b) (2 PTS.)

```
var message = switch (result) {  
    case Maybe.Some(var value) ->  
        "first negative: " + value;  
  
    case Maybe.None<?> _ ->  
        "no negative value";  
};
```

(c) (2 PTS.)

The first call returns Some(-3). The second returns None. Maybe<Integer> represents either a found value or the absence of one.

7. (10 PTS.) This question deals with a command-line MVC application for a playlist.

```
public record Song(String title, int seconds) { }

public record Playlist(VList<Song> songs) {
    public VList<Song> longerThan(int seconds) {
        return songs.filter(s -> s.seconds() > seconds);
    }
}
```

(a) (4 PTS.)

```
class CliView {
    public void showSongs(VList<Song> songs) {
        for (var song : songs) {
            IO.println(song.title());
        }
    }
}
```

(b) (3 PTS.)

```
class Controller {
    private Playlist model;
    private CliView view;

    public void showLongSongs(int seconds) {
        var songs = model.longerThan(seconds);
        view.showSongs(songs);
    }
}
```

(c) (3 PTS.)

The controller is taking on presentation/output responsibility by printing the song titles itself. That belongs to CliView in this application. The controller should delegate the result to the view:

```
public void showLongSongs(int seconds) {
    view.showSongs(model.longerThan(seconds));
}
```

The inferred type of songs in Timmy's version is VList<Song>.